

Communication Efficient Gradient Coding: A Survey

Jay Mehta: 22B1281
Kshitij Vaidya: 22B1829
Tanay Bhat: 22B3303

May 6, 2026

Contents

1	Implementation	2
1.1	Setup	2
1.2	Results	3

Chapter 1

Implementation

In this chapter, we will discuss our own implementation of a toy example of a gradient coding scheme. We will implement a simple gradient coding scheme based on the idea of polynomial interpolation as described in the reference paper. We will use Python for our implementation.

1.1 Setup

We have made an assumption in our implementation.

$$n \text{ (Number of workers)} = k \text{ (Number of datasets)}$$

This is consistent with the reference paper, which also makes a similar assumption for simplicity.

We test the correctness of our implementation by trying out multiple test cases mentioned below:

Test Case	n = k	d	s	m	l
1	5	3	1	2	4
2	6	3	1	2	6
3	8	4	2	2	8
4	10	5	3	2	20
5	10	4	2	2	20
6	10	3	1	2	20
7	12	6	3	3	12
8	15	5	2	3	15

Table 1.1: Test cases for our implementation

n = number of workers, k = number of datasets, d = number of datasets assigned to each worker, s = number of stragglers tolerated, m = communication reduction factor, l = dimension of the gradient vector.

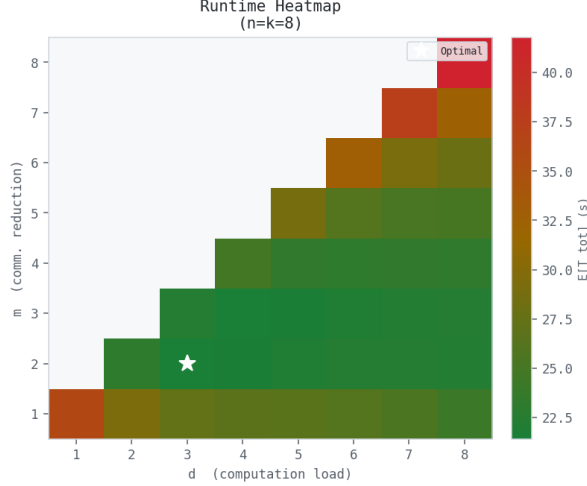
Once these test cases are passed, we can be confident that our implementation is correct and can be used for further experiments and analysis. After this we try to model the computation and communication time as shown in section 5 of the reference paper. We plot the modeled expected running times for our code and compare it with naive distributed schemes. The errors in the gradient recovery are also plotted for different values of n , s and m .

Values used for various plots:

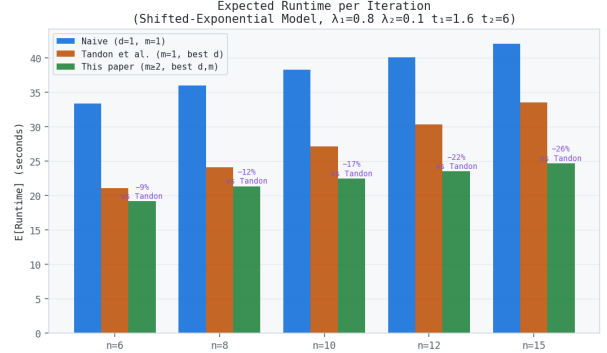
- $n = k = 8$ for runtime heatmap
- For expected runtime plot, we use $d = 1$, $m = 1$ for naive scheme, $m = 1$ and best d for $m = 1$ based on heatmap for Tandon scheme and $m \geq 2$ and best d , m pair from heatmap for the proposed scheme. Also $\lambda_1 = 0.8$, $\lambda_2 = 0.1$ and $t_1 = 1.6$, $t_2 = 6$ for the expected runtime.
- For communication cost vs straggler tradeoff plot, we use $n = 12$ and vary d and s values.

- For gradient error vs s plot, we use $n = 10$, $l = 20$ and vary s and m values, $d = s + m$ in all cases.

1.2 Results

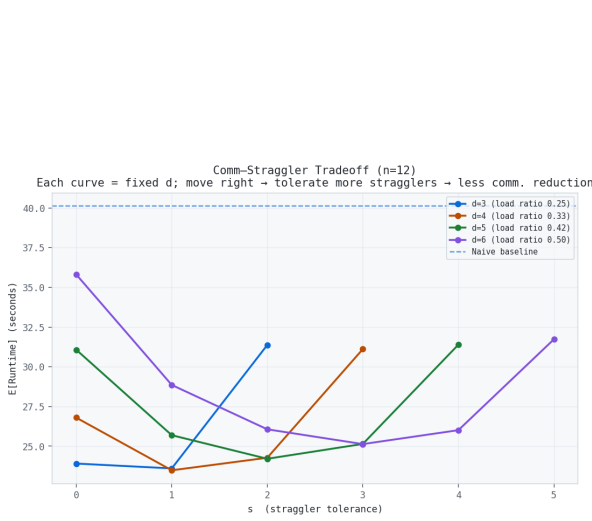


(a) Runtime heatmap for different m , d values for $n = 8$

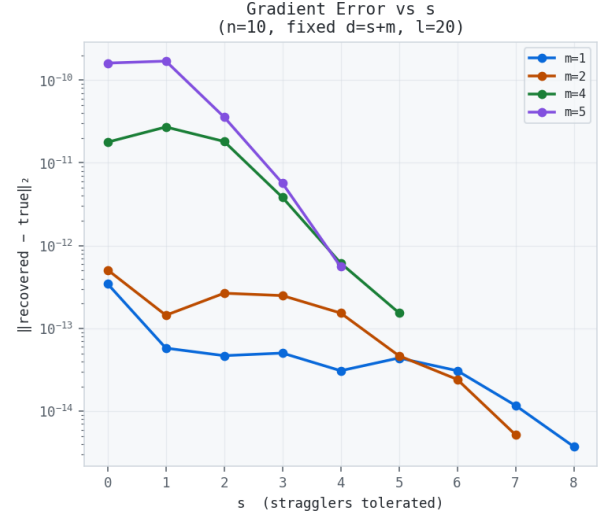


(b) Expected runtime for different schemes

As we can see, the first plot shows the runtime heatmap for different values of m and d for $n = 8$. The second plot shows the expected runtime for different schemes, where we can see that the proposed scheme has a lower expected runtime compared to the naive scheme and Tandon scheme.



(a) Communication cost vs straggler tradeoff



(b) Gradient error vs s

The first plot shows the communication cost vs straggler tradeoff for $n = 12$, where we can see that as we increase the number of stragglers tolerated, the communication cost decreases initially and then increases after a certain point. The second plot shows the gradient error vs s for $n = 10$, $l = 20$ and different m values, where we can see that as we increase the number of stragglers tolerated, the gradient error decreases and it is higher for higher m values for same s values.